EXP-6. Error Correction at Data link layer.

- Aim:

Write a program to implement error detection and correction using HAMMING code concept. Make a test run to input data stream & verify error correction feature.

- Description:

Hamming code is a set of error-correction codes that can be used to detect and correct the errors that can occur when data is transmitted from sender to receiver.

- Procedure:

Sender program:

① Input to sender file should be a text of any length program should convert the text to binary.

② Apply hamming code concept on the binary data and add redundant bits to it

③ Save this output in a file called channel.

Receiver program:

① Receiver program should read the input channel file.

② Apply hamming code on the binary data to check for errors

③ If there is error, display the position of error.

④ Else remove the redundant bits & convert the binary data to ascii & o/p.

CODE:

andu.py

```
import sys

def encode_nibble_hamming(d):
    d1 = (d>>3)&1
    d2 = (d>>2)&1
    d3 = (d>>1)&1
    d4 = (d&1)

    p1 = d1^d2^d4
    p2 = d1^d3^d4
    p3 = d2^d3^d4

    code = (
        (p1<<6)|(p2<<5)|(d1<<4)|(p3<<3)|
                        (d2<<2)|(d3<<1)|
                                d4)

    return format(code, "07b")

def encode_character(ch):
    ascii_val = ord(ch)
    high_nibble = (ascii_val >> 4) & 0x0F
    low_nibble = ascii_val & 0x0F
    return encode_nibble_hamming(high_nibble)
            + encode_nibble_hamming
                (low_nibble)
```

```
def main():
    text = input()
    encoded_str = "".join(encode_char(ch)
    print(encoded_stream)
    with open("channel", "w") as f:
        f.write(encoded_stream)

if __name__ == "__main__":
    main()
```

receiver.py

```
with open("channel") as f:
    s = f.read().strip()

chars = []
for i in range(0, len(s), 14):
    byte_val = 0
    for nibble_bits in (s[i:i+7], s[i+7:i+14]):
        b = [0] + [int(x) for x in nibble_bits]
        err = (
            ((b[4] ^ b[5] ^ b[6] ^ b[7]) << 2) |
            ((b[2] ^ b[3] ^ b[6] ^ b[7]) << 1) |
            (b[1] ^ b[3] ^ b[5] ^ b[7])
        )
        if err:
            b[err] ^= 1
        d = (b[3] << 3) | (b[5] << 2) | (b[6] << 1) | b[7]
        byte_val = (byte_val << 4) | d
    chars.append(chr(byte_val))
```

- Output:

Enter text msg : REC
Original text : REC.
Binay with hamming
011100100110010111001011011010111001000101 1

Reciever section.
Recieved bits.
011100 10011001011100101101101011100100010 11

Final decoded Message : REC

- Result:

Hence the python code for implementing Hamming code at the data link layer. were successfully executed and verified